

JWT, Testing & Docker

Securing APIs, Testing Like a Pro & Containerizing Your App 🚀

TOPICS



```
1 from fastapi import FastAPI
2 from auth import create_token
3
4 @app.post("/login")
5 def login(user: User):
6     # 1. Validate User
7     # 2. Return JWT
8     return {"token": create_token(user)}
```

docker-compose.yml

```
1 services:
2   api: { image: "backend:latest" }
3   db: { image: "mysql:8.0" }
```

Agenda



JWT Authentication

Securely managing user identity and access control.

01



Testing APIs

Validating endpoints with pytest and TestClient.

02



Docker Basics

Containerizing applications for consistent deployment.

03

What is JWT?

JSON Web Token (RFC 7519) is a compact, URL-safe means of representing claims to be transferred between two parties.

</> ENCODED TOKEN STRUCTURE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

1 Header

METADATA

Contains the type of token (JWT) and the signing algorithm being used, such as HMAC SHA256 or RSA.

```
{ "alg": "HS256", "typ": "JWT" }
```

2 Payload

DATA (CLAIMS)

Contains the claims. Claims are statements about an entity (typically, the user) and additional data.

```
{ "sub": "123", "name": "John" }
```

3 Signature

VERIFICATION

Used to verify the message wasn't changed along the way. Created by hashing the header, payload, and a secret.

```
HMACSHA256(base64(head) + "." + ...)
```

JWT Structure

1. HEADER // Algorithm & Token Type

```
01  {  
02    "alg": "HS256",  
03    "typ": "JWT"  
04  }
```

2. PAYLOAD // Claims (Data)

```
05  {  
06    "sub": "user_123", // Subject (User ID)  
07    "name": "Ahmed", // Custom Claim  
08    "iat": 1712342078, // Issued At (Timestamp)  
09    "exp": 1712345678 // Expiration Time  
10  }
```

3. SIGNATURE // Verification Logic (Pseudo-code)

```
11  HMACSHA256(  
12    base64UrlEncode(header) + "." +  
13    base64UrlEncode(payload),  
14    "your-256-bit-secret"  
15  )
```



AUTHENTICATION PROCESS

How JWT Works

Stateless authentication flow between Client and Server

1



Login

Client sends credentials (username & password) to the server.

```
POST /login
{ user, pass }
```

2



Issue Token

Server validates creds, creates JWT, signs it with SECRET.

```
sign(claims, secret)
return token
```

3



Storage

Client receives token and stores it safely (LocalStorage or Cookie).

```
localStorage
.setItem('token', t)
```

4



API Call

Client sends token in Authorization header for subsequent requests.

```
Authorization:
Bearer eyJhbG...
```

5



Verify

Server verifies signature & expiration. Grants access if valid.

```
if valid:
return data
```



JWT in FastAPI

terminal & config.py

1. INSTALLATION // Install required packages

```
$ pip install fastapi uvicorn python-jose[cryptography] passlib[bcrypt] python-multipart
```

2. CONFIGURATION CONSTANTS // Key Security Variables

```
01 import os
02
03 # Generate a secure key: openssl rand -hex 32
04 SECRET_KEY = "09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf63b88e8d3e7"
05
06 # Algorithm used to sign the JWT
07 ALGORITHM = "HS256"
08
09 # Token expiration time
10 ACCESS_TOKEN_EXPIRE_MINUTES = 30
```



Creating JWT Tokens

python-jose • datetime • Security

utils.py

IMPORTS

```
01 from datetime import datetime, timedelta
02 from jose import jwt
```

FUNCTION DEFINITION

```
04 def create_access_token(data: dict, expires_delta: timedelta | None = None):
05     to_encode = data.copy()
```

EXPIRATION & ENCODING

```
07 if expires_delta:
08     expire = datetime.utcnow() + expires_delta
09 else:
10     expire = datetime.utcnow() + timedelta(minutes=15)
11
12 to_encode.update({"exp": expire}) # Add expiration claim
13 encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
14 return encoded_jwt
```


Protecting Routes

Dependency Injection • OAuth2PasswordBearer

main.py

```
1. DEFINE SCHEME // Tells FastAPI where to find the token

01 from fastapi.security import OAuth2PasswordBearer
02 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

2. DEPENDENCY // Decodes token & returns user or raises 401

03 async def get_current_user(token: str = Depends(oauth2_scheme)):
04     try:
05         payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
06         username = payload.get("sub")
07         if username is None: raise HTTPException(status_code=401)
08         return username
09     except JWTError:
10         raise HTTPException(status_code=401, detail="Invalid token")

3. PROTECT ENDPOINT // Inject dependency into route

11 @app.get("/users/me")
12 async def read_users_me(user: str = Depends(get_current_user)):
13     return {"username": user, "status": "authenticated"}
```




What is API Testing?

Validating logic, reliability, and security of endpoints



Why Test APIs?



Prevent Regressions

Ensure new features don't break existing functionality. Catch bugs early in the dev cycle.



Validate Contracts

Guarantee endpoints return expected data structures, status codes, and headers.



Security & Performance

Verify auth flows work correctly and APIs respond within acceptable time limits.

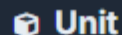


Living Documentation

Tests serve as accurate examples of how to use the API for other developers.



Test Hierarchy



Unit

Test individual functions in isolation (fastest).



Integration

Test interaction with DB & other services.



E2E

Simulate full user flows from start to finish.



Load

Stress test for concurrency & stability.



The Toolkit



pytest



TestClient



httpx



coverage.py





TESTING FRAMEWORK

pytest Basics

Simple • Scalable • Pythonic

terminal & test_basics.py

1. SETUP // Installation & Naming Convention

```
01 $ pip install pytest
02 # Naming: Files must start with "test_*.py" or end with "*_test.py"
```

2. THE CODE // Writing a simple test function

```
03 def test_simple_math():
04     value = 10
05     assert value + 5 == 15 # Standard Python assert
06     assert value > 0
07
```

3. EXECUTE // Run from terminal

```
08 $ pytest -v
09 ... tests/test_basics.py::test_simple_math PASSED [100%]
```



FastAPI TestClient

pytest • httpx • requests-style

tests/test_main.py

```
01 from fastapi.testclient import TestClient
02 from app.main import app
03
04 # Create a client instance wrapping the FastAPI app
05 client = TestClient(app)
06
07 def test_health_check():
08     # Send GET request to /health endpoint
09     response = client.get("/health")
10
11     # Assertions
12     assert response.status_code == 200
13     assert response.json() == {"status": "ok"}
```

Key Feature

No Server Needed

TestClient builds on httpx to call your FastAPI app directly in Python. It doesn't need to start a real server (like uvicorn) or use network ports.



Testing with Authentication

tests/test_auth_routes.py

```
01 def test_read_users_me(client):  
    1. AUTHENTICATE // Login to retrieve access token  
02     response = client.post(  
03         "/token",  
04         data={"username": "johndoe", "password": "secret"}  
05     )  
06     token = response.json()["access_token"]  
  
    2. AUTHORIZED REQUEST // Use token in Authorization header  
07     headers = {"Authorization": f"Bearer {token}"}  
08  
09     response = client.get("/users/me", headers=headers)  
10  
11     assert response.status_code == 200  
12     assert response.json()["username"] == "johndoe"
```



Test Fixtures & conftest.py

tests/conftest.py

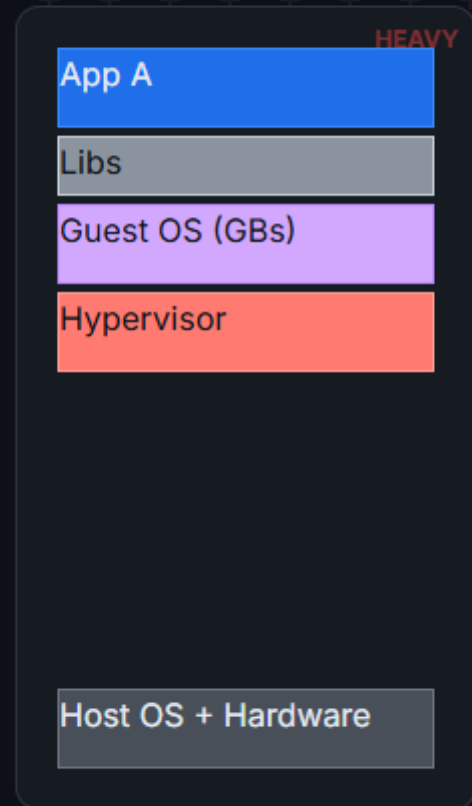
```
01 import pytest
02 from fastapi.testclient import TestClient
03 from app.main import app
04
05 @pytest.fixture
06 def client():
07     # Create a fresh TestClient for each test
08     return TestClient(app)
09
10 @pytest.fixture
11 def auth_headers(client):
12     # Reusable authentication logic
13     resp = client.post("/token", data={"username": "alice", "password": "123"})
14     token = resp.json()["access_token"]
15     return {"Authorization": f"Bearer {token}"}
16
```



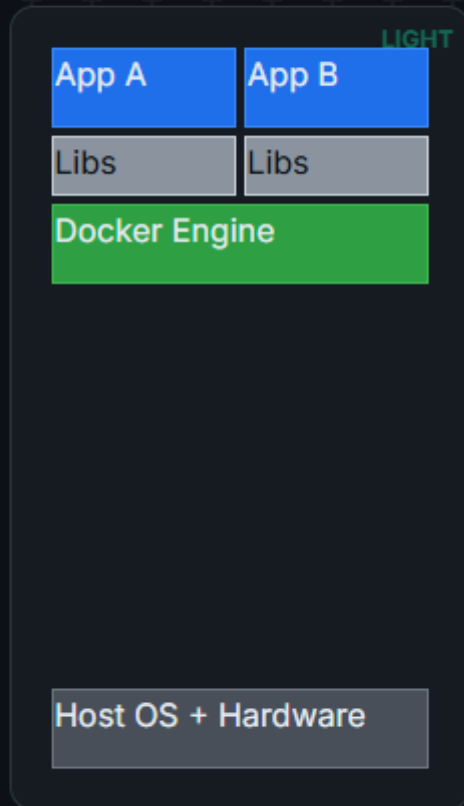
What is Docker?

Architecture Battle

Virtual Machine



Container



A platform to develop, ship, and run applications by packaging them into standardized units called containers.

3 Core Concepts



Image

The Blueprint

A read-only template with instructions for creating a Docker container. Think of it as a **snapshot** or a Class in OOP.



Container

The Instance

A runnable instance of an image. It is isolated from other containers and the host machine. Think of it as an Object in OOP.



Dockerfile

The Recipe

A text document that contains all the commands a user could call on the command line to assemble an image.



Dockerfile for FastAPI



Dockerfile

```
01  # 1. Use official lightweight Python image
02  FROM python:3.11-slim
03  # 2. Set the working directory inside the container
04  WORKDIR /app
05  # 3. Copy requirements and install dependencies
06  COPY requirements.txt .
07  RUN pip install --no-cache-dir -r requirements.txt
08  # 4. Copy the rest of the application code
09  COPY . .
10  # 5. Document the port the app runs on
11  EXPOSE 8000
12  # 6. Command to run the application
13  CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```




Docker Commands Cheat Sheet

Essential CLI operations for your daily workflow

Build & Lifecycle



Build Image

```
docker build -t myapp:v1 .
```

Creates an image from Dockerfile in current directory.



Run Container

```
docker run -p 8000:8000 myapp:v1
```

Starts a container mapping host port 8000 to container 8000.



Stop Container

```
docker stop <container_id>
```

Gracefully stops a running container.



Execute Command

```
docker exec -it <id> sh
```

Opens an interactive shell inside a running container.



Inspect & Manage



List Containers

```
docker ps -a
```

Shows all running and stopped containers.



View Logs

```
docker logs -f <container_id>
```

Follows the log output of a container in real-time.



Pull Image

```
docker pull python:3.11-slim
```

Downloads an image from Docker Hub registry.



Remove Image

```
docker rmi <image_id>
```

Deletes an image from local storage (use -f to force).



MULTI-SERVICE ORCHESTRATION

Docker Compose.

```
01  version: "3.9"
02  services:
    APP SERVICE // FastAPI Backend
03  app:
04  build: . # Build from current Dockerfile
05  ports: ["8000:8000"]
06  env_file: .env
07  depends_on: [db] # Wait for DB to start

    DB SERVICE // MySQL Database
08  db:
09  image: mysql:8.0
10  environment:
11  - MYSQL_DATABASE=app_db
12  - MYSQL_ROOT_PASSWORD=secret
13  volumes:
14  - db_data:/var/lib/mysql

    PERSISTENCE // Named Volume
15  volumes:
16  db_data:
```



Environment Variables

Project Configuration

 .env

// Secrets & Config (Don't commit this!)

```
01 DATABASE_URL="mysql+pymysql://root:secret@db:3306/app"
02 SECRET_KEY="super-secret-change-me-123"
```



docker-compose.yml

// Injecting into containers

```
03 services:
04   app:
05     build: .
06     env_file: # Loads variables from .env file automatically
07     - .env
```

 config.py

// Validation with Pydantic

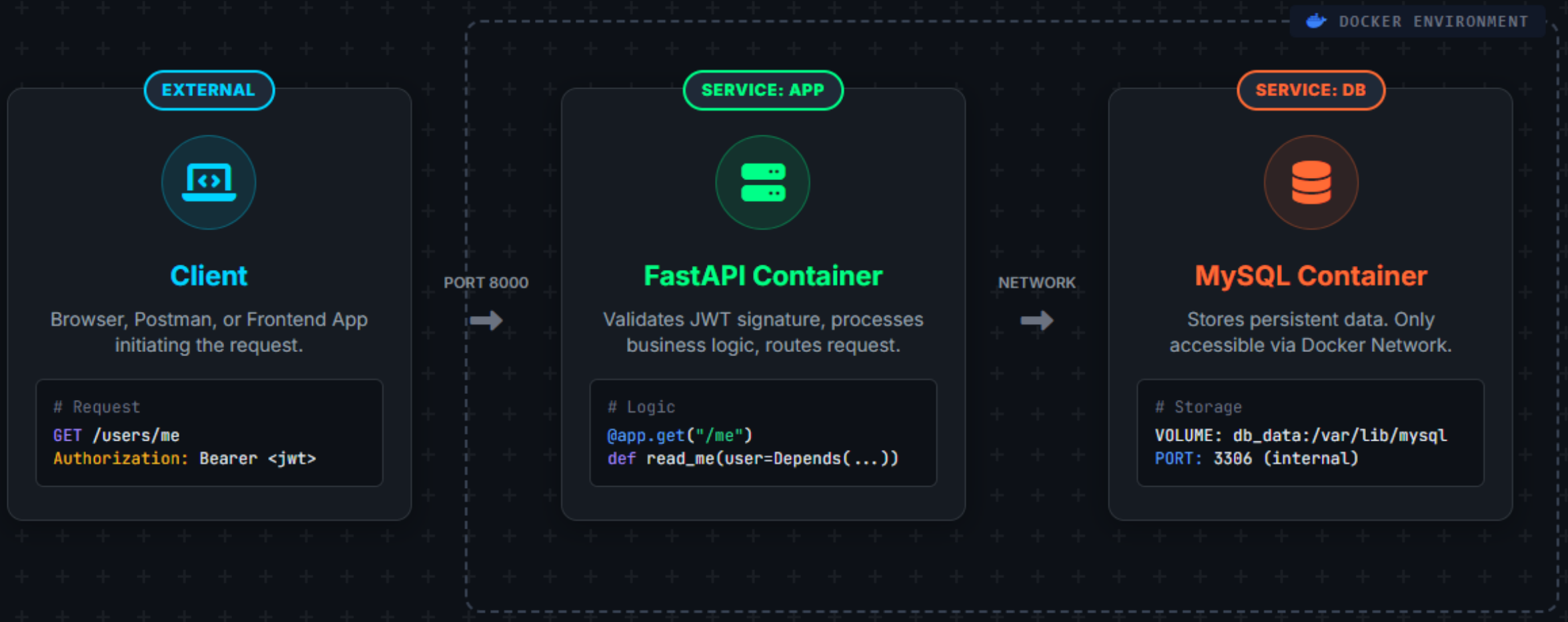
```
08 from pydantic_settings import BaseSettings
09
10 class Settings(BaseSettings):
11     database_url: str
12     secret_key: str
13
14 class Config:
```

```
    env_file = ".env" # Reads .env for dev (if no env vars set)
```



Full Architecture Overview

Client-Server interaction within a containerized Docker environment





RECAP & REFERENCE

Summary & Best Practices

Key takeaways for building secure, testable, containerized apps



JWT Security

AUTH & PROTECTION

- ✓ **Short Expiration:** Keep access tokens short-lived (e.g., 15-30 mins) to limit exposure.
- ✓ **Secure Storage:** Never store tokens in LocalStorage if XSS is a risk; prefer HttpOnly cookies.
- ✓ **Strong Secrets:** Generate complex secret keys and rotate them periodically.
- ✓ **HTTPS Only:** Always transmit tokens over encrypted SSL/TLS channels.



Testing Strategy

PYTEST & QUALITY

- ✓ **Isolated DB:** Use a separate database (e.g., SQLite in-memory) for tests to protect real data.
- ✓ **Fixtures:** Leverage `conftest.py` for reusable setup logic like clients and auth headers.
- ✓ **High Coverage:** Aim for >80% code coverage, focusing on critical business logic.
- ✓ **Mock External APIs:** Don't hit real 3rd party services; mock responses to ensure stability.



Docker Optimization

DEVOPS & DEPLOY

- ✓ **Lightweight Images:** Use `python:slim` or Alpine variants to reduce container size.
- ✓ **Multi-stage Builds:** Separate build dependencies from the runtime environment.
- ✓ **.dockerignore:** Exclude files like `__pycache__`, `.git`, and `env` vars.
- ✓ **Non-root User:** Run applications as a non-privileged user inside the container.

>_ QUICK COMMANDS

\$ `pytest -v` // Run Tests

\$ `docker compose up -d --build` // Start App

\$ `docker logs -f app_container` // Monitor